Merge Sort

Merge sort is a [[Divide and Conquer]] sorting algorithm that recursively sorts halves and merges them.

How It Works (Natural Explanation)

The strategy is: - Split the array into two halves. - Sort each half. - Merge the two sorted halves into one sorted result.

The key idea is that merging two sorted arrays is easy and linear, so the algorithm spends effort on repeated splitting plus linear merges.

Formal View

For array A of size n: - If $n \leq 1$, return A. - Recursively sort left half and right half. - Merge the two sorted halves in linear time.

The merge step maintains two pointers and repeatedly outputs the smaller front element.

Why The Time Complexity Is What It Is

Let $T(n)$ be runtime for size n. - Two recursive calls on size $n/2$: $2T(n/2)$. - One merge over all n elements: $O(n)$.

So:

$$T(n) = 2T(n/2) + O(n).$$

This solves to:

$$T(n) = O(n \log n).$$

Intuition: - There are about $\log_2 n$ recursion levels. - Each level does total linear merge work $O(n)$. - Total = $O(n)$ per level times $O(\log n)$ levels.

Best, average, and worst case are all $O(n \log n)$.

Space Complexity

- Typical array implementation uses $O(n)$ extra memory for merging.

When To Use It

- You need predictable $O(n \log n)$ performance.
- Stability is required.
- Linked-list variants are especially effective.

Strengths And Limitations

- Strengths:
  - Strong asymptotic performance.
  - Stable sorting.
  - Runtime does not degrade on bad input order.
- Limitations:
  - Extra memory overhead in common array implementations.

Related

- [[Insertion Sort]]
- [[Divide and Conquer]]
- [[Recurrence Relations]]
- [[Sorting Algorithms]]
- [[Lectures/Week 2]]